

ChallengeDaily

Security Audit Report

Architecture: Electron + React + Python Flask SQLite

Audit Date: 2026-07-08

Scope: Full source code, 15 security dimensions

Findings: 1 CRITICAL / 5 HIGH / 8 MEDIUM / 6 LOW

Findings Summary

ID	Severity	Dimension	File	Line	Description
F-01	CRITICAL	Command Injection	system_events.py	155-168	PowerShell script f-string injection
F-02	HIGH	Input Validation	settings_routes.py	68-70	ai_base_url no protocol check
F-03	HIGH	DoS Risk	profile.py	23-31	User profile fields no length limit
F-04	HIGH	Race Condition	db.py / activities.py	199-218	Soft delete non-atomic
F-05	HIGH	Electron Security	main.cjs	165-169	PATH in child env enables DLL hijack
F-06	HIGH	Command Injection	main.cjs	512	ExecutionPolicy Bypass for geo
F-07	MEDIUM	Auth/Authorization	notifications.py	32-37	No auth on notification read+mark
F-08	MEDIUM	Input Validation	pomodoro.py	10-26	No duration/category validation
F-09	MEDIUM	Backup Security	backup.py	96-158	Temp file may leak on crash
F-10	MEDIUM	Prompt Injection	chat.py / activities.py	57-64	Manual activity summary unfiltered
F-11	MEDIUM	Input Validation	activities.py	298-303	Timeline no date validation
F-12	MEDIUM	Input Validation	profile.py	69-77	Daily profile no date validation
F-13	MEDIUM	Input Validation	week_plan.py	23-51	Path params no format check
F-14	MEDIUM	Auth/Authorization	app_rules.py	112-133	Icon endpoint unauthenticated
F-15	LOW	Input Validation	stats.py	227-232	Greeting params inject AI prompt
F-16	LOW	DoS Risk	week_plan.py	123-140	Meta fields no length limit
F-17	LOW	DoS Risk	diaries.py	20-37	Diary content no length limit
F-18	LOW	Key Management	config.py	35	Settings JSON exposes AI config
F-19	LOW	Data Residue	webhooks.py	17	Webhook URLs stored in plaintext
F-20	LOW	DoS Risk	collector.py	29	Idle threshold hardcoded

F-01 CRITICAL — PowerShell Command Injection

File: system_events.py:155-168 | **Dimension:** Command Injection

Description: get_boot_events() and get_login_events() embed user-controllable start_date/end_date parameters directly into PowerShell script strings via f-string. While current callers use validate_date(), the module itself performs no validation. If a future caller bypasses validation, arbitrary PowerShell commands can be injected.

Evidence: Attack vector: end_date = "2026-01-01"); Start-Process calc; #" → arbitrary code execution.

Fix: 1. Add force regex validation `^\d{4}-\d{2}-\d{2}$` inside system_events.py functions. 2. Use -ArgumentList parameter passing instead of string interpolation.

F-02 HIGH — ai_base_url No Protocol Validation

File: settings_routes.py:68-70 | **Dimension:** Input Validation

Description: POST /api/settings accepts ai_base_url without protocol validation. Backup restore validates http/https, but the settings API does not. Attackers can set file:/// or gopher:// URLs for SSRF.

Evidence: config.AI_BASE_URL = str(data['ai_base_url']) — no scheme check.

Fix: Add protocol check matching backup.py's validation: must start with http:// or https://.

F-03 HIGH — User Profile No Length Limits

File: routes/profile.py:23-31 | **Dimension:** DoS Risk

Description: save_profile() accepts arbitrary-length JSON fields (role_desc, work_style, habits, custom_rules) without size limits. 10MB custom_rules would bloat the database and cause AI prompt overflow.

Evidence: All fields from data dict written directly to DB via save_user_profile(data).

Fix: Add per-field length limits: role_desc/work_style max 500 chars, custom_rules max 50 items, etc.

F-04 HIGH — Soft Delete Non-Atomic Race Condition

File: routes/activities.py:199-218 | **Dimension:** Race Condition / Database Integrity

Description: Soft delete performs SELECT → INSERT into deleted table → DELETE from activities in separate execute() calls within the same connection. Another thread's operation between SELECT and INSERT could cause inconsistent state. SQLite with check_same_thread=False permits concurrent access.

Evidence: Lines 162-218: conn.execute(SELECT), then conn.execute(INSERT), then conn.execute(DELETE).

Fix: Use BEGIN IMMEDIATE transaction to make the entire SELECT+INSERT+DELETE atomic.

F-05 HIGH — Child Process PATH Enables DLL Hijack

File: client/electron/main.cjs:165-169 | **Dimension:** Electron Security

Description: Backend Python subprocess inherits PATH from parent. If an attacker modifies PATH order (placing a malicious directory first), Python may load a trojaned DLL or python.exe itself.

Evidence: allowedEnvKeys includes 'PATH' in the whitelist.

Fix: In packaged mode, remove PATH dependency and use explicit embedded Python path. Validate pythonExe is within expected directory.

F-06 HIGH — ExecutionPolicy Bypass for Geolocation

File: client/electron/main.cjs:512 | **Dimension:** Electron Security / Command Injection

Description: Geolocation uses -ExecutionPolicy Bypass to run a PowerShell script. While script content is hardcoded (no user input), this flag violates enterprise group policies. The temp script file (cd_geo_ + 8 random hex bytes) is written to %TEMP% and could theoretically be pre-created by an attacker.

Evidence: powershell -NoProfile -ExecutionPolicy Bypass -File "\${tmpScript}"

Fix: 1. Use -ExecutionPolicy RemoteSigned instead of Bypass. 2. Use -EncodedCommand to avoid writing temp files. 3. Increase random suffix to 16 bytes (32 hex chars).

F-07 MEDIUM — Notification Auto-Read Without Auth Binding

File: routes/notifications.py:32-37 | **Dimension:** Auth/Authorization

Description: GET /api/notifications automatically marks all unread notifications as read upon access. No user session binding — any request with a valid token can read and clear notifications. Notifications may contain sensitive data (daily report summaries). Not persisted to disk — lost on process restart.

Evidence: with _notifications_lock: unread = [n for n in _notifications if not n['read']]; for n in unread: n['read'] = True

Fix: 1. Separate read and mark-read endpoints. 2. Persist notifications to SQLite. 3. Rate-limit notification read requests.

F-08 MEDIUM — Pomodoro Missing Input Validation

File: routes/pomodoro.py:10-26 | **Dimension:** Input Validation

Description: start_pomodoro() accepts duration_min with no range check (negative or 999999) and category with no enum validation (unlike todos_VALID_CATEGORIES).

Evidence: duration_min = int(data.get('duration_min', 25)); category = data.get('category', '■■■')

Fix: Add: duration_min = max(1, min(120, int(...))); if category not in CATEGORIES: category = '■■■'.

F-09 MEDIUM — Backup Temp File Leak on Exception

File: routes/backup.py:96-158 | **Dimension:** Backup/Restore Security

Description: Uploaded ZIP saved to temp file. The os.unlink(tmp_path) at line 156 is inside the main try block. If an exception occurs between lines 98 and 156, the temp file is leaked. Process crash also leaves the file.

Evidence: with tempfile.NamedTemporaryFile(suffix='.zip', delete=False) as tmp: ... # unlink inside try, not finally

Fix: Move os.unlink to a finally block to guarantee cleanup.

F-10 MEDIUM — Manual Activity Summary Unfiltered for AI Injection

File: routes/chat.py:57-64, activities.py:74-118 | **Dimension:** Prompt Injection

Description: Activities created via POST /api/activities store user-provided summary directly. These summaries are later loaded by build_weekly_context() and injected into AI system prompts. Unlike collector.py's _sanitize_title(), manual summaries bypass filtering.

Evidence: Manual: db.insert_manual_activity(summary=summary); AI context: build_weekly_context() reads all activities.

Fix: Apply _sanitize_title() or equivalent filtering to the summary field in create_activity().

F-11 MEDIUM — Timeline Endpoint No Date Validation

File: routes/activities.py:298-303 | **Dimension:** Input Validation

Description: /api/timeline passes startDate and endDate directly to get_activities() without validate_date(). Not SQL injection (parameterized), but invalid input causes unexpected behavior.

Evidence: start = request.args.get('startDate', date.today().isoformat())

Fix: Add validate_date() check matching other endpoints.

F-12 MEDIUM — Daily Profile Generate No Date Validation

File: routes/profile.py:69-77 | **Dimension:** Input Validation

Description: POST /api/profile/daily//generate passes date_str to generate_daily_profile() without format validation. GET for same path also lacks validation.

Evidence: def generate_daily_profile(date_str): profile = _gen(date_str)

Fix: Add validate_date() in route handler before calling generator.

F-13 MEDIUM — Week Plan Path Params No Format Check

File: routes/week_plan.py:23-51 | **Dimension:** Input Validation

Description: month_key and week_start URL path parameters passed directly to DB queries without format validation. Should be YYYY-MM and YYYY-MM-DD respectively.

Evidence: def get_month(month_key): tasks = db.get_month_tasks(month_key)

Fix: Add regex validation: month_key ^\d{4}-\d{2}\$, week_start use validate_date().

F-14 MEDIUM — Icon Endpoint Unauthenticated

File: routes/app_rules.py:112-133 | **Dimension:** Auth/Authorization

Description: /api/icons/ is public (no token required). The app_name parameter is sanitized for file system operations (safe_name replaces \ / :), but the raw app_name still reaches icon_extractor's _find_exe_fallback() which scans the filesystem. Potential for abuse with malformed names.

Evidence: Public prefix in server.py: _PUBLIC_PREFIXES = ['/api/icons/']

Fix: 1. Add character whitelist validation for app_name (alphanumeric + . + - + _). 2. Consider moving behind auth.

F-15 LOW — Greeting Query Params Inject AI Prompt

File: routes/stats.py:227-232 | **Dimension:** Input Validation

Description: /api/greeting passes URL query params (time, date, weekday, lunar, location, weather, temp) directly into AI context. Token-protected but prompt injection possible if token is compromised.

Evidence: context = {'time': request.args.get('time', ''), ...}

Fix: Add length limits and character whitelist for all query params before injecting into AI prompt.

F-16 LOW — Week Plan Meta No Length Limit

File: routes/week_plan.py:123-140 | **Dimension:** DoS Risk

Description: title and goal fields in PUT /api/week-plan/meta have no length limit. Could store extremely long strings.

Evidence: db.update_plan_meta(plan_type, plan_key, title=data.get('title', ''), goal=data.get('goal', ''))

Fix: Limit title to 200 chars, goal to 1000 chars.

F-17 LOW — Diary Content No Length Limit

File: routes/diaries.py:20-37 | **Dimension:** DoS Risk

Description: Diary content, tags, highlights, gratitude fields have no length limit. Theoretically allows GB-scale text insertion.

Evidence: db.upsert_diary(content=data.get('content', ''), ...)

Fix: Set reasonable limits: content max 50000 chars, other fields 2000 chars.

F-18 LOW — Settings JSON Exposes AI Configuration

File: config.py:35 | **Dimension:** Key Management

Description: settings.json stores ai_base_url, ai_vision_model, ai_text_model in plaintext. While API keys are DPAPI-encrypted, AI service provider and model names are exposed.

Evidence: SETTINGS_PATH = DATA_DIR / 'settings.json'

Fix: Low priority. Consider encrypting AI-related settings fields with DPAPI.

F-19 LOW — Webhook URLs Stored in Plaintext

File: routes/webhooks.py:17 | **Dimension:** Sensitive Data Residue

Description: Webhook URLs containing secret tokens are stored in plaintext webhooks.json. Logs are sanitized but disk file is not.

Evidence: _WEBHOOK_PATH = BASE_DIR / 'data' / 'webhooks.json'

Fix: Encrypt webhook URLs with DPAPI before storage, decrypt on read.

F-20 LOW — Idle Threshold Hardcoded

File: collector.py:29 | **Dimension:** DoS Risk / Privacy

Description: _IDLE_THRESHOLD_SEC = 180 (3 minutes) is hardcoded. Too small → missing data; too large → privacy risk of continued capture after user leaves.

Evidence: _IDLE_THRESHOLD_SEC = 180

Fix: Move to settings.json as configurable option, range-limited to 60-600 seconds.